

TASK 1 - Password Vault (Frontend Only)

Overview

In this task, you will build a **password vault**, i.e., a page where you can generate passwords, save them, and "share" one with a simulated one-time link. There's no backend and no login yet, everything is stored in your browser's [localStorage](#).

Step 1: Set up your pages and routing

Before writing any vault logic, get navigation working first.

Create four routes using React Router:

- [/login](#) — just a form UI for now, doesn't need to do anything yet.
- [/signup](#) — same, just a form UI for now.
- [/](#) (or [/vault](#)) — this will be your main Vault Dashboard, where most of the work happens.
- [/share-preview/:id](#) — a page that will later show a single password entry.

At this stage, all four pages can just render a heading like "Login Page" so you can click between them and confirm routing works before building anything real inside them.

Concept used: [React Router](#) (basic setup + navigation) -> reactrouter.com

Step 2: Build the password generator

This is a self-contained piece. Build and test it on its own before connecting it to anything else.

Make a form with:

- A slider or number input for password length.
- Checkboxes for: uppercase letters, lowercase letters, numbers, symbols.
- A "Generate" button.
- A box that displays the generated password, with a copy button next to it.

Write your own function that takes the selected options and returns a random password (do not use a library for this). Use [crypto.getRandomValues\(\)](#) for actual randomness, not [Math.random\(\)](#).

At this point, generating a password should just show it on screen. You don't need to save it anywhere yet.

Concepts used: [useState](#) (for the options and result), [controlled forms](#), secure random generation → developer.mozilla.org ([Crypto.getRandomValues](#))

Step 3: Add the ability to save an entry

Now build a second small form below the generator. This one has a label input and a password field.

The password field should be fillable two ways:

1. Typed in manually.
2. Filled automatically by clicking a "Use generated password" button, which pulls in whatever the generator last produced.

This second option is the tricky part. The generated password lives in one component's state, but you need to use it in a different form. This means moving that piece of state up to a shared parent component so both the generator and this form can access it.

Concepts used: *useState, controlled forms, lifting state up* -> react.dev/learn/sharing-state-between-components

Step 4: Save entries to localStorage (this is where your custom hook comes in)

Right now, if you refresh the page, everything you typed disappears. Fix this by writing your own custom hook and call it `useVault` or `useLocalStorage`.

This hook should:

- Load saved entries from `localStorage` when the component first mounts.
- Automatically save entries back to `localStorage` whenever they change.

Don't use a library for this either; writing it yourself is the point.

Concepts used: *useEffect (for the load-on-mount and save-on-change behavior), custom hooks* → react.dev/learn/reusing-logic-with-custom-hooks

Step 5: Display the list of saved entries

Now render every saved entry as a row: label, password (masked by default, e.g. ●●●●●●), a button to reveal/hide it, a copy button, and a delete button.

Two things to be careful about here:

- When generating a unique ID for each entry, use `crypto.randomUUID()` (do NOT use the array index, since deleting an entry would then mix up which row is which).
- Handle the empty state: if there are no entries yet, show a message instead of an empty list.

Concepts used: *lists & keys, conditional rendering, props (passing each entry down to its row)*

Step 6: Build the Share flow

Add a "Share" button to each entry that navigates to `/share-preview/:id`, using that entry's ID.

On the Share Preview page:

- Read the `:id` from the URL.
- Look up that entry.
- If this is the FIRST time this id has been viewed, show the password and mark it as "viewed" in `localStorage`.
- If it's already been viewed before, show "already viewed" instead of the password.

Concepts used: React Router (reading dynamic `:id` params), React Router (navigation), conditional rendering

Step 7 (optional bonus): Password strength meter

If you have time left, add a small function that scores a password based on its length and character variety, and show that score next to the generator or the saved entry.

Useful Tips

- If something isn't saving after refresh, check your custom hook's `useEffect` dependency array first
- Use DevTools -> Application -> Local Storage to watch what your app is actually saving, in real time
- `useState`'s starting value is only read once, ever. All updates after that come from calling the setter function, not from `useState` re-running
- If your initial state takes real work to compute (like reading `localStorage`), wrap it in a function: `useState(() => ...)`, not `useState(...)`
- Two nearby components needing the same data? Lift that state up to their shared parent, then pass it down as props
- A value/function can be renamed when destructured. `setValue` inside a hook and `setEntries` outside it can be the exact same function under two names
- Always use `crypto.randomUUID()` for list item IDs, never the array index; deleting items will otherwise scramble which row is which
- `key` props exist so React can track *which* item is which across re-renders, not just *how many* there are

Deliverable

By the end of this task, you should have a working local-only password vault: you can generate a password, save it with a label, see it in a list (masked, with reveal/copy/delete), and share it via a link that only shows the password once. Login and Signup exist as pages but don't do anything real yet.

Resources for Inspo

- JSX Components - [Every React Concept Explained in 12 Minutes](#)
- Controlled forms - [Full React Tutorial #27 - Controlled Inputs \(forms\)](#) , [React – Sharing State](#)
- List and keys - [Lists and keys in React | React Tutorials](#) , [React - Rendering Lists](#)
- Conditional rendering - [React 19 Tutorial - 10 - Conditional Rendering](#) , [React – Conditional Rendering](#)
- Lifting state up - [Lifting State up in ReactJs Tutorial #24 | Complete React Course](#) , [React – Sharing State](#)
- Random password - [Create a random password generator in JavaScript](#)
- Use navigate - [#81: useNavigate Hook in React Router: Seamless Page Redirection & Back ...](#) , [React Router – useNavigate](#)
- Use params - [Learn useParams Hook in React in 5 Minutes | Hooks in React | useParams](#) , [React Router – useParams](#)
- React router - [React Router tutorial | react-router-dom in 5 minutes](#) , [React Router – Official Docs](#)
- Clipboard API - [JavaScript Clipboard API Programming Tutorial](#) , [MDN – Clipboard.writeText](#)
- Component Communication & Core State - [PROPS in React explained](#) 
- Managing Side Effects - [Learn React Hooks: useEffect - Simply Explained!](#)
- Custom Hooks & Architecture - [Custom Hooks in React \(Design Patterns\)](#) , [Learn Custom Hooks In 10 Minutes](#)

UI/UX for inspo

1. Bitwarden - vault dashboard, masked/unmasked entries, copy buttons.
2. Dashlane - reference for a clean "share" flow.
3. NordPass - minimal dashboard layout.